

MixBytes()

1inch Aqua & Swap-VM Security Audit Report

AUGUST 12, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Assessment Methodology	6
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	11
2.1 Critical	11
2.2 High	11
2.3 Medium	11
M-1 Aqua protocol can be permanently waived by the maker, causing protocol revenue loss	11
2.4 Low	13
L-1 _onlyTakerTokenSupplyShareGte overflows and fails closed on large-supply tokens	13
L-2 A self-referential fee recipient collects nothing and emits no ProtocolFeeSkipped	14
L-3 _onlyTxOriginTokenBalanceNonZero does not bind the party that trades	15
L-4 The documented PeggedSwap example sets linearWidth to effectively zero	16
3. About MixBytes	17

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

SwapVM is a virtual machine for programmable token swaps: instead of deploying a contract per strategy, makers compose bytecode programs from a fixed instruction set, and the VM executes them on demand to price and settle a trade. The scoped instructions cover pricing curves ([PeggedSwap](#) – a square-root linear curve for pegged assets, backed by the analytical solver in [PeggedSwapMath](#)), execution guards ([Controls](#) – deadlines, conditional jumps, and taker/[tx.origin](#) token-holding gates), and fee collection ([Fee](#) – flat, protocol, and dynamic-provider fees charged on [amountIn](#)), with [AquaOpcodes](#) binding them into the opcode table exposed to Aqua-mode strategies. Two settlement modes coexist: signature-based orders, where tokens move directly between maker and taker wallets, and Aqua-mode orders, where maker liquidity is held in the external Aqua shared-liquidity layer and moved via [push/pull](#) against a per-strategy balance. The principal actors are makers, who author and fund strategies, and takers, who quote and execute against them.

The interim security review was conducted over 1 working day via manual code review and a proprietary AI-assisted analysis tool.

Because the scoped instructions execute against the external Aqua liquidity layer and can consult third-party fee-provider contracts, our focus covered the core instruction logic together with the boundaries where it hands control to those external components. Alongside that, the team worked through the usual internal checklist – access control, reentrancy, arithmetic overflow and rounding direction, and the handling of untrusted return data.

The Aqua shared-liquidity layer and any external protocol fee provider contracts reachable through the dynamic fee instructions were not themselves subject to line-by-line review in this engagement; we examined only the call sites in the scoped code and the assumptions those calls rest on, relying on the dependencies' published interfaces for the rest.

This engagement follows the earlier MixBytes review of Aqua & Swap-VM, published as the [linch Aqua & Swap-VM Security Audit Report](#).

The audited commit is the head of release [v1.0.2](#). That release changes exactly one source file against v1.0.1: [src/instructions/Fee.sol](#), which introduces the best-effort Aqua fee path.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **The instruction set is disciplined, but several documented guarantees are not enforced where they are consumed.** Each instruction is small, single-purpose, and carries NatSpec stating its argument layout and ordering constraints, with guard rails where they matter: `_feeAmountIn` refuses to run after amounts are computed, and `PeggedSwap` rejects recomputation. The gap is that some stated rules live only in off-chain builder helpers – the `feeBps <= BPS` bound is checked there and not in the on-chain parsers, and the overflow bounds documented throughout `PeggedSwapMath` are assumed rather than validated in `PeggedSwapArgsBuilder.parse`. We recommend moving each such check to the point where the value enters the contract.
- **Best-effort fee collection is the dominant risk in this scope.** `Fee._tryPullFee` wraps the Aqua fee pull in a bare `try/catch`, so an uncollectable fee is skipped and reported via `ProtocolFeeSkipped` instead of reverting. This fixes a real liveness defect, but the trigger is the maker's own wallet balance and Aqua allowance, and pricing is not adjusted when the pull fails, so the taker still pays a fee that never reaches the protocol. We recommend distinguishing a discretionary shortfall from a genuinely one-sided position so the liveness fix does not double as a costless fee-evasion path.
- **The event that best-effort fee collection depends on cannot be filtered on-chain.** `ProtocolFeeSkipped` declares no `indexed` parameters, so it carries a single topic – its own signature – and `eth_getLogs` cannot filter it by strategy, token or recipient.
- **The 63/64 gas rule does not give a taker control over fee collection.** Because the pull sits inside a `try/catch`, only 63/64 of the remaining gas is forwarded to it, which in principle lets a caller starve a subcall while the outer frame survives. Sweeping the gas supplied to `swap()` from 100,000 to 1,200,000, across 430 successful swaps we found no window in which the swap settled and the fee was skipped: the pull runs early in `runLoop`, so any limit low enough to starve it also starves the heavier settlement that follows and the whole transaction reverts. Fee collection therefore fails only on maker-controlled conditions, which is what keeps the finding below scoped to the maker.
- **The unbounded `feeBps` in the non-dynamic fee opcodes remains open from the previous audit.** The on-chain parsers `parseFlatFee` and `parseProtocolFee` apply no bound, so a hand-encoded program can set any value; the dynamic variants do check it. At `feeBps == BPS` the exact-out denominator is zero and the swap reverts with a bare panic, above `BPS` both directions underflow, and at `feeBps == BPS - 1` an exact-out swap settles while demanding $1e9$ times the intended input from a taker who omits a threshold. The team acknowledged this in the previous audit ("Fee division by zero when `feeBps == BPS` in exact-out"), declining to add a check that passes for every non-erroneous order and noting that takers must validate strategies before use. We have not re-raised it, but record that the behaviour is unchanged and the taker-side threshold remains the only protection.
- **A higher linear width makes a depeg worse, and the permitted ceiling is far above the useful range.** In $\sqrt{u} + \sqrt{v} + A(u + v) = C$, raising `A` grows the linear term against the square-root term, flattening the curve toward a constant-sum line and removing the curvature that slows an outflow during a depeg. The ceiling is a raised one: `MAX_LINEAR_WIDTH` was introduced as `500 * ONE` and lifted tenfold to `5000 * ONE`. We recommend documenting this where `linearWidth` is parsed, and reverting the ceiling toward the configurations the curve is intended for.
- **Maker-authored bytecode is trusted by construction.** Jump targets are unvalidated, so a program can jump into the middle of another instruction's arguments and execute bytes that a linear reading does not show, and unbounded loops are expressible. Takers cannot rewrite a program, so this is not a taker-side attack, but a strategy's disassembly is not by itself evidence of its behaviour. Whitelisting should therefore reason about executed paths rather than instruction listings.

1.3 Project Overview

Summary

Title	Description
Client	linch
Category	DEX Aggregator
Project	Aqua & Swap-VM
Type	Solidity
Platform	EVM
Timeline	28.07.2026 - 03.08.2026

Scope of Audit

File	Link
src/instructions/Fee.sol	src/instructions/Fee.sol
src/instructions/Controls.sol	src/instructions/Controls.sol
src/instructions/PeggedSwap.sol	src/instructions/PeggedSwap.sol
src/libs/PeggedSwapMath.sol	src/libs/PeggedSwapMath.sol
src/opcodes/AquaOpcodes.sol	src/opcodes/AquaOpcodes.sol

Versions Log

Date	Commit Hash	Note
28.07.2026	32c687c2b73101fc26549e48fa1ff8a4d73afbac	Initial Commit

Mainnet Deployments

Deployment verification will occur after the smart contracts are deployed on the mainnet.

1.4 Security Assessment Methodology

Stage 1: Interim Audit

Project Architecture Review

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM, HOW THEY INTERACT ARCHITECTURALLY, AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

Activities include:

- understanding overall system design and contract interactions
- mapping responsibilities of each component and protocol workflows
- conducting a thorough line-by-line review of contracts and modules
- tracing execution paths and mapping state transitions
- identifying trust boundaries and external integrations
- forming an independent architectural understanding directly from code before validating documentation
- running the project test suite to observe implementation behavior and encoded invariants
- reviewing documentation, specifications, READMEs, and deployment guides and reconciling them with code

Adversarial Code Review

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

Approach includes:

- analyzing the codebase from an attacker's perspective, focusing on what can fail rather than only confirming intended behavior
- searching for overlooked edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- attempting to violate protocol invariants and identifying scenarios where system guarantees break
- analyzing economic attack surfaces including liquidation logic, oracle dependencies, vault accounting, and cross-contract interactions
- writing targeted tests, modelling adversarial scenarios, applying mutation testing and fuzzing, and developing proof-of-concept exploits
- conducting independent exploit path discovery by each auditor to reduce anchoring bias
- reviewing high-risk code collaboratively in pair-auditing sessions led by the Team Lead

Systematic Vulnerability Analysis

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

Activities include:

- reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- analyzing common DeFi attack classes such as reentrancy, accounting manipulation, oracle manipulation, privilege escalation, and state desynchronization
- using proprietary AI tooling to surface candidate issues across broader attack vectors
- manually validating and triaging AI-generated candidates

Consolidation of Auditors' Reports

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

Activities include:

- cross-checking findings across auditors
- consolidating and deduplicating issues
- resolving discrepancies in severity assessments
- using proprietary AI tooling to assist with report drafting and consistency checks
- manually reviewing and finalizing the report narrative
- preparing the interim audit report for client review

Stage 2: Re-audit & Mainnet Deployment Verification

Re-audit

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THAT THE UPDATED CODE DOES NOT INTRODUCE NEW VULNERABILITIES.

Activities include:

- confirming each remediation matches the original recommendation
- documenting rationale for any unresolved findings
- verifying modified logic and integration points remain secure
- executing the project's tests after remediation, including targeted coverage of fixed areas
- using proprietary AI tooling on the updated codebase to surface potential new issues

Mainnet Deployment Verification

OBJECTIVE: PERFORM FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC AUDIT REPORT.

Activities include:

- verifying deployed contract bytecode against audited source code across networks
- confirming deployed bytecode matches the build produced from the audited commit using identical compiler settings
- reviewing constructor and initializer arguments used in deployment
- verifying proxy deployment order, initialization flow, and admin configuration
- ensuring implementations are not left uninitialized or exposed to initialization front-running
- publishing the final report after alignment between client and audit team

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.
- **Medium** – Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.
- **Low** – One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

Likelihood

- **High** – An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g., due to a market condition that the actor cannot influence).
- **Medium** – An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the contract owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed in order to minimize potential risks.
- **Medium** – Recommended to be fixed to improve security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and efficiency.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and the issue no longer impacts the security of the protocol.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but the issue has not been fully resolved.
- **Acknowledged** – The recommended fixes have not been implemented, and the finding remains unresolved or has been accepted by the project team without code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	1
Low	4

Findings Statuses

ID	Finding	Severity	Status
M-1	Aqua protocol can be permanently waived by the maker, causing protocol revenue loss	Medium	Acknowledged
L-1	<code>_onlyTakerTokenSupplyShareGte</code> overflows and fails closed on large-supply tokens	Low	Acknowledged
L-2	A self-referential fee recipient collects nothing and emits no <code>ProtocolFeeSkipped</code>	Low	Acknowledged
L-3	<code>_onlyTxOriginTokenBalanceNonZero</code> does not bind the party that trades	Low	Acknowledged
L-4	The documented PeggedSwap example sets <code>linearWidth</code> to effectively zero	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

M-1	Aqua protocol can be permanently waived by the maker, causing protocol revenue loss		
Severity	Medium	Status	Acknowledged

Description

This issue has been identified within the `_tryPullFee` function of the `Fee` contract (reached through the `_aquaProtocolFeeAmountInXD` and `_aquaDynamicProtocolFeeAmountInXD` instructions). The pricing is intentionally left unchanged when the pull fails. Because `amountNetPulled` is credited only on success, the settlement accounting in `SwapVM._transferIn` (`balanceIn >= originalAquaBalanceIn + amountIn - amountNetPulled`) then requires the taker to push the full, fee-inclusive `amountIn`, while the uncollected fee simply remains with the maker. The taker therefore always pays the fee, but on a failed pull the protocol receives nothing and the maker keeps it as a windfall.

The critical property is that the maker fully controls whether the pull succeeds, and can do so even after the strategy has passed any admission/whitelisting review. `AQUA.pull` moves the fee out of the maker's own wallet via `safeTransferFrom(maker, to, amount)` and additionally decrements the maker's virtual Aqua balance. Consequently the maker can force the pull to revert at any time, invisibly, through several independent and reversible levers:

- **Revoking the allowance after whitelisting.** A strategy can be admitted while the maker holds a sufficient `tokenIn` allowance to the Aqua contract, and the maker can later submit a plain `approve(AQUA, 0)`.
- **ERC777 callback.** If `tokenIn` were an ERC777-style token with transfer callbacks, the maker could register a `tokensToSend` hook that reverts selectively (e.g. only when `to` is the protocol collector), forcing a deterministic skip even while fully funded.

The issue is classified as **Medium** severity because it allows a permissionless and easily reproducible loss of protocol fee revenue together with an equal maker windfall on every affected swap.

Recommendation

We recommend collecting the protocol fee after the taker's `tokenIn` has been transferred in – when the incoming balance is available to fund it.

Client's Commentary:

Acknowledged by design, will be fixed in version 1.2; design choice before next release, correct fix requires deep changes in architecture

2.4 Low

L-1	<code>_onlyTakerTokenSupplyShareGte</code> overflows and fails closed on large-supply tokens		
Severity	Low	Status	Acknowledged

Description

`Controls.sol:148` evaluates the share check as unguarded checked arithmetic:

```
require(totalSupply > 0 && balance * 1e18 >= minShareE18 * totalSupply, ...);
```

Both products can overflow independently. The left-hand side, `balance * 1e18`, reverts once `balance` exceeds `type(uint256).max / 1e18`, so a taker holding 100% of supply against a 1% threshold reverts even though the check should plainly pass. The boundary is exact: at `balance == type(uint256).max / 1e18` the expression still evaluates, and one wei higher reverts. The right-hand side, `minShareE18 * totalSupply`, overflows independently of the balance, so with `totalSupply == type(uint256).max` any `minShareE18` of 2 or more reverts even for a small balance, and a maker cannot avoid the problem by choosing a lower threshold. The issue is classified as Low severity because it affects liveness only, leaving the order unfillable for the takers concerned without exposing funds, and it requires a gate token whose supply or balances reach the extreme end of the uint256 range.

Recommendation

We recommend changing the calculation to `balance >= Math.mulDiv(minShareE18, totalSupply, 1e18)`, which is exact and cannot overflow for any input.

Client's Commentary:

Acknowledged by design, could happen only to very big supply or some exotic tokens; planned to use only with good audited tokens and related to a liveness of a strategy which can be easily reshipped

L-2	A self-referential fee recipient collects nothing and emits no <code>ProtocolFeeSkipped</code>		
Severity	Low	Status	Acknowledged

Description

`_aquaProtocolFeeAmountInXD` validates only that the recipient is non-zero (`Fee.sol:137`). Setting `to` to the maker makes `Aqua.pull` perform a maker-to-maker transfer, which succeeds, so the fee is treated as collected.

The maker does not get the fee for free. `Aqua.pull` debits the strategy's Aqua balance at `Aqua.sol:66` before the transfer, so that balance ends at the same value as with an honest recipient (measured: 1009.9e18 in both cases); only the maker's wallet delta differs, because the transfer itself is a no-op. The fee therefore moves from the strategy's committed inventory into the maker's freely held tokens rather than being created.

`amountNetPulled` and the real debit cancel exactly in the `SwapVM.sol:240` pre-push check, so the taker cannot underpay either.

The protocol receives nothing, yet no `ProtocolFeeSkipped` is emitted. `Fee.sol:65-67` describes that event as "the only signal that protocol revenue was not collected", so a self-referential recipient is indistinguishable on-chain from a healthy fee-paying strategy and is invisible to the monitoring this design depends on.

The issue is classified as Low severity because the protocol collects nothing while the strategy remains indistinguishable on-chain from a healthy fee-paying one, but the maker gains only a transfer from committed to free inventory rather than new value, takers are unaffected, and it requires the maker to configure the recipient this way.

Recommendation

Validate the fee recipient when a strategy is whitelisted, rejecting a `to` equal to the maker. An on-chain `require(to != ctx.query.maker)` in the Aqua fee instructions would make the case unrepresentable, at the cost of one comparison per swap.

Client's Commentary:

Acknowledged by design. Actually fee is not skipped, and paid, even if the result is zero; back can detect it

L-3	<code>_onlyTxOriginTokenBalanceNonZero</code> does not bind the party that trades		
Severity	Low	Status	Acknowledged

Description

`_onlyTxOriginTokenBalanceNonZero` gates a strategy on `IERC20(token).balanceOf(tx.origin)` (`Controls.sol:122-127`). Its NatSpec notes that NFTs are natively supported, and the natural use for a non-transferable token gate of this kind is a KYC or allowlist credential. `tx.origin` is the EOA that started the transaction, not the party the trade is for. Any intermediate contract unrelated to the protocol – an aggregator, router or batch executor – passes the gate as long as its own caller happens to hold the credential, so a cleared user can front a transaction whose economic beneficiary holds nothing. We confirmed that a gated strategy executes under a credentialed `tx.origin` and that the proceeds then reach an address with no credential, with nothing in the protocol observing the final recipient. It also excludes parties it should admit. Under EIP-4337 there is no meaningful `tx.origin` for the account: it is the bundler. A smart account holding the credential – which is where a KYC issuer would mint it – fails the gate outright, because the bundler holds nothing. We confirmed this reverts with `TxOriginTokenBalanceIsZero(bundler, credential)`. The defect is specific to the `tx.origin` variant rather than to gating in general. `_onlyTakerTokenBalanceNonZero` checks `ctx.query.taker`, and in the same smart-account scenario it correctly admits the credentialed account regardless of who submitted the transaction. The issue is classified as Low severity because it requires a maker to select this opcode for a credential gate and grants no direct access to funds, but where it is used for KYC the gate provides an appearance of control it does not deliver.

Recommendation

Gate on `ctx.query.taker` via `_onlyTakerTokenBalanceNonZero`, which binds the trading party and is unaffected by account abstraction. If the `tx.origin` variant is retained, document that it constrains only the transaction signer and must not be relied on for compliance, and state that quoting such a strategy requires an explicit `from`.

Client's Commentary:

ACK, by design, selected by Maker who should totally understand how tx.origin works

L-4	The documented PeggedSwap example sets <code>linearWidth</code> to effectively zero		
Severity	Low	Status	Acknowledged

Description

The linear width `A` is what gives the pegged curve its flatness near parity, and the instruction's own guide recommends `A ≈ 100e27-300e27` for tight stablecoin pairs (`PeggedSwap.sol:127`).

The repository's single worked example sets it far below that range. `test/RunLoop.t.sol:284` specifies `linearWidth: 0.02e9` under the comment "2% width for stable pairs". The field is scaled by `1e27`, so `2e7` is an `A` of roughly `2e-20`. We measured the curve at this setting returning byte-identical output to `A = 0`, so the configured value is not merely small but operationally absent: the strategy is a plain square-root curve with none of the flatness the parameter exists to provide. Every other call site in the repository uses the correct scale.

The example is maker-facing. `docs/PROGRAMS.md:266` reproduces the snippet verbatim as a recommended composition, so a maker following the documentation for a stablecoin pair deploys the untuned curve. The same snippet also sets `x0/y0` to `50e18` against a pool initialised at `100e18`, contradicting the normalisation convention stated at `PeggedSwap.sol:17` and followed everywhere else.

Neither value is caught by the test suite. `test_BestRouteSelector_XYC_vs_Pegged` asserts only that the output falls in `0.98e18 < amountOut < 1e18`, and with `1e18` into its `100e18` pool the `XYC` branch returns `0.990099e18`, the curve as written `0.995025e18` and the curve as intended `0.999975e18` – all three inside the band, so it cannot discriminate between them. The `test/invariants/pegged/` suite sweeps `A` across five files, but every property it checks holds for any valid `A`, so it establishes that the curve is self-consistent rather than that it is the curve that was configured. We confirmed this by mutation: `pegged/BalancedCurve.t.sol` passes all nine of its tests unchanged when its `linearWidth` is replaced with `0.02e9`.

The issue is classified as Low severity because it affects documentation and test coverage rather than protocol logic, and a maker who deploys the untuned curve gets a functioning square-root AMM rather than a broken or exploitable one, but the deployed strategy will not behave as the documentation says it does.

Recommendation

Correct `linearWidth` and the `x0/y0` values in `test/RunLoop.t.sol` and in the copy at `docs/PROGRAMS.md`, and add coverage that pins the behaviour of a specific `A` so that a wrong value fails a test rather than passing silently.

Client's Commentary:

ACK, the scale factor will be updated next release

3. About MixBytes

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems.

The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle – from early architecture design to production audits and ongoing protocol evolution.

Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including **Lido**, **Aave**, **Curve**, **linch**, **OKX**, **Mantle**, **Fluid**, **Gearbox**, **Resolv**, and others, helping teams identify vulnerabilities, strengthen protocol design, and improve the robustness of decentralized systems.

To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review. These tools augment the work of senior auditors but do not replace expert analysis.

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

- **Design Review.** Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.
- **AI Tooling.** AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.
- **Smart Contract Audit.** Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.
- **Security Retainer.** Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

Contacts



<https://mixbytes.io/>



https://github.com/mixbytes/audits_public



<https://x.com/MixBytes>



MixBytes()



hello@mixbytes.io